

ՀԱՅԱՍՏԱՆԻ ԱԶԳԱՅԻՆ ՊՈԼԻՏԵԽՆԻԿԱԿԱՆ ՀԱՄԱԼՍԱՐԱՆ

Ինտեգրալ սխեմաների նախագծման
օգնականի մշակում և իրականացում

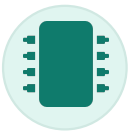
Խումբ՝	SS218
Ուսանող՝	Նատալյա Գոլչենկո
Ղեկավար՝	Արման Մանուկյան

Խումբ՝	SS261
Ուսանող՝	Արտաշես Գրիգորյան
Ղեկավար՝	Դավթյան Վաչագան



Աշխատանքի Նպատակը

Ստեղծել **արհեստական բանականությամբ աշխատող ծրագրային օգնական**, որն ապահովում է թվային սխեմաների System Verilog նկարագրման լեզվով գեներացիա և ստուգում



System Verilog Նկարագրություններ

Ստեղծում է սխեմաների և թեստ-բենչերի ճշգրիտ նկարագրություններ՝ կախված օգտատիրոջ ընտրած պարամետրերից ու ֆունկցիոնալ պահանջներից

Ֆունկցիոնալ գեներացիա



Սեփական Տվյալների Բազա

Օգտագործում է մասնագիտացված, կուտակված գիտելիքների բազա՝ ճշգրիտ ու հուսալի արդյունքների և արդյունավետ աշխատանքի ապահովման համար

Ներքին ռեսուրս



Որոնմամբ Հզորացված Գեներացիա

Կիրառում է RAG տեխնոլոգիա՝ տեղեկատվության որոնման միջոցով ելքային արդյունքի որակն ու ճշտությունը բարձրացնելու համար

RAG արխիտեկտուրա



Խնդրի դրվածք



ԱԲ Համակարգերի Ուսումնասիրություն

01

Արհեստական բանականությամբ համակարգերի և արդի թվային սխեմաների խորը ուսումնասիրություն, առկա մոտեցումների վերլուծություն

ԱԲ • Արդի սխեմաներ



Սխեմաների Նախագծում և Թեստավորում

02

Արդի թվային սխեմաների System Verilog նկարագրությունների ստեղծում, ֆունկցիոնալ թեստավորման միջավայրերի մշակում

System Verilog • Testbench



Բազայի Հավաքագրում և Մշակում

03

Սխեմաների, պարամետրերի և ֆունկցիոնալ նկարագրությունների հավաքագրում, կառուցվածքավորում, ստեղծում և մշակում

Տվյալների բազա



ԱԲ Համակարգի Մշակում

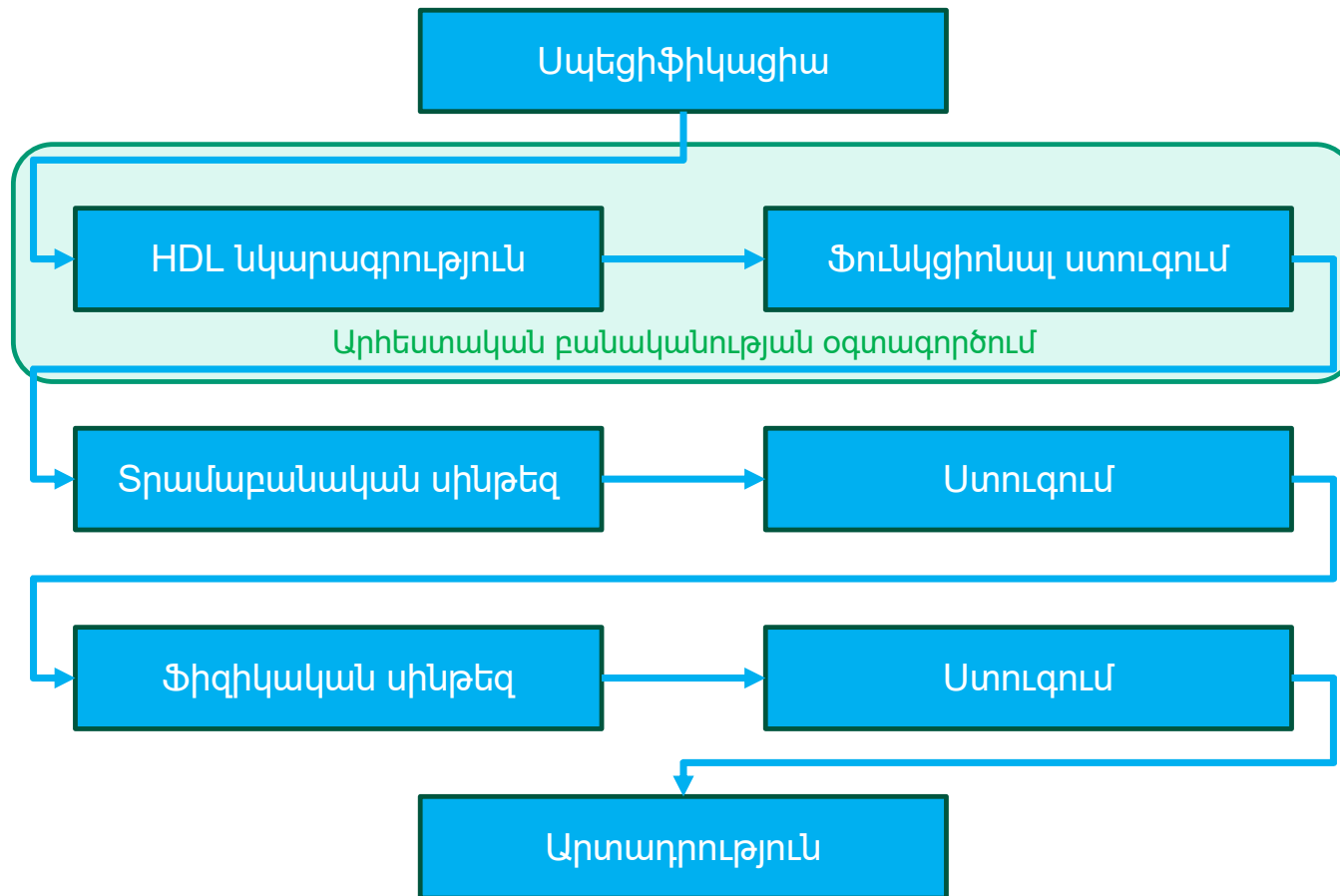
04

Բազայի հիման վրա արհեստական բանականությամբ աշխատող համակարգի մշակում, ինտեգրում և փորձարկում

RAG • ԱԲ օգնական



Ներածություն(մաս 1)



Ներածություն(մաս 2)

ԱԲ-ի դերը HDL նկարագրությունում

- Բնական լեզվով նկարագրությունից HDL կոդի գեներացում
- Կոդի կառուցվածքի բարելավում և պարզեցում
- Կրկնվող մոդուլների արագ ստեղծում
- Շարահյուսական սխալների հայտնաբերում և ուղղում

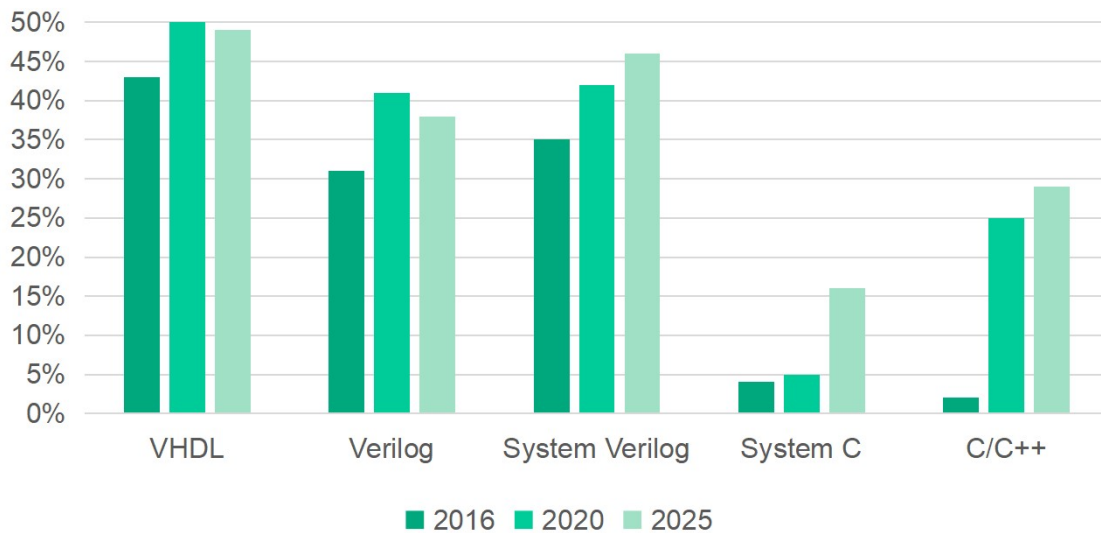
ԱԲ-ի դերը Ֆունկցիոնալության ստուգման փուլում

- testbench-երի ավտոմատ գեներացում
- Մուտքային տարբեր սցենարների ստեղծում
- Տրամաբանական սխալների հայտնաբերում
- Մոդուլի ելքերի համեմատում սպասվող արդյունքների հետ
- Սխալների վերլուծություն և ուղղում

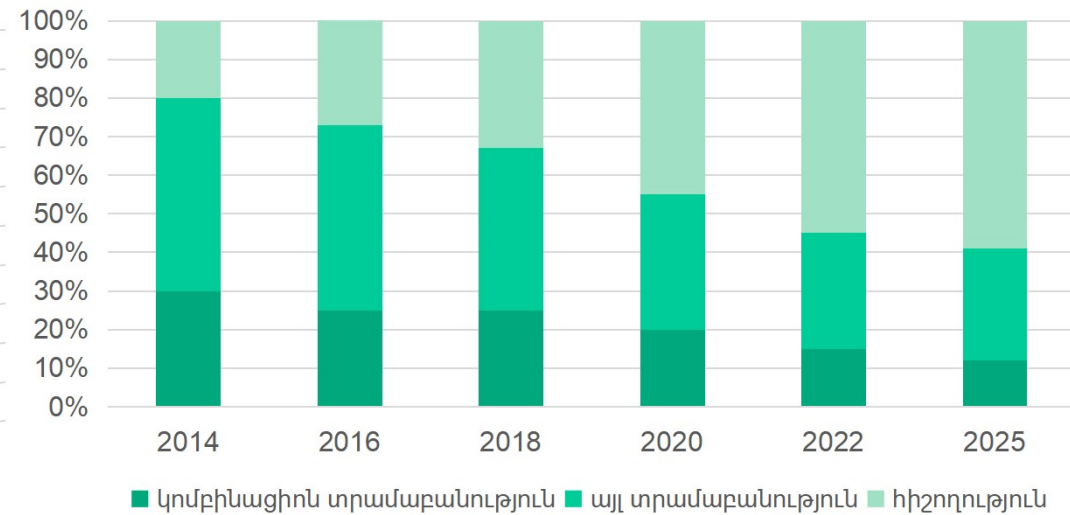


Արդիականություն

Միսեմաների նկարագրման լեզուների կիրառությունը



ԻՍ-երի բաղադրիչների կիրառությունները



- Foster H. 2024 Wilson Research Group FPGA functional verification trends.
- 2024 International Technology Roadmap for Semiconductors (ITRS).

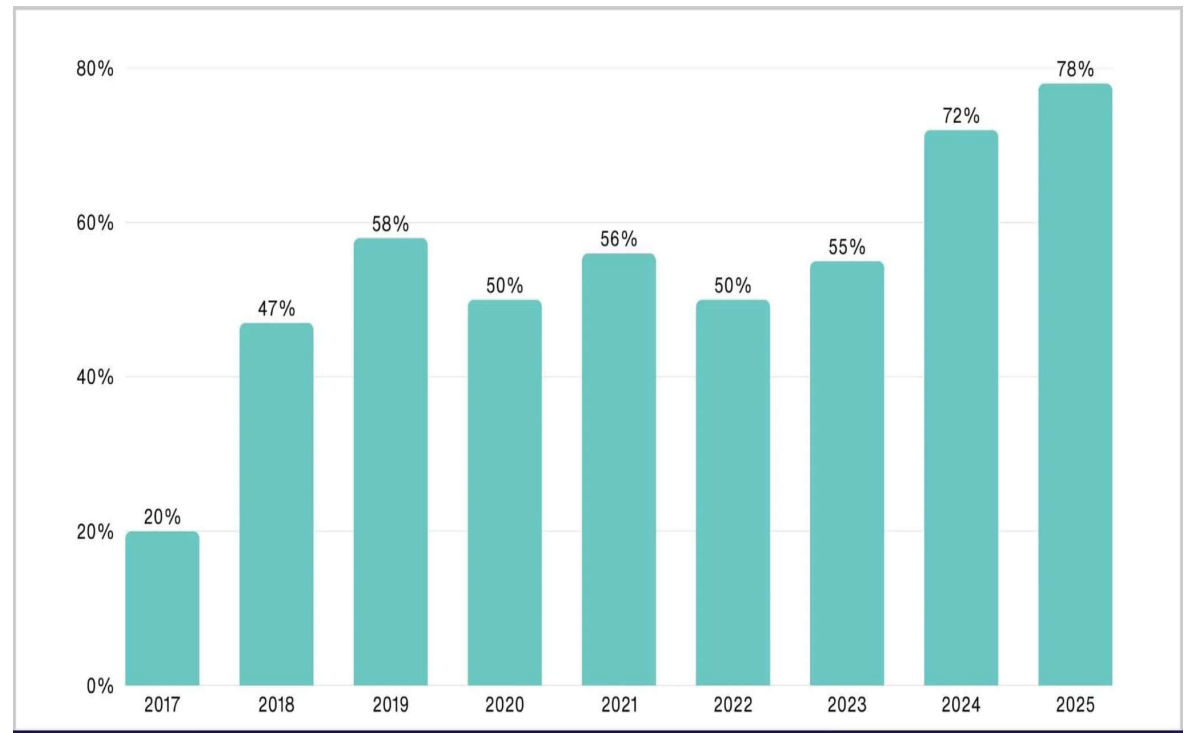


Արհեստական Բանականության Արդիականությունը

ԱԲ-ի կիրառման աճը ընկերություններում

Ընկերությունների կողմից ԱԲ տեխնոլոգիաների կիրառման աճը տարեցտարի հիմնված համաշխարհային շուկայի ուսումնասիրությունների վրա:

Շարունակական աճը պայմանավորված է ավտոմատացման, տվյալների վրա հիմնված որոշումների կայացման և մրցակցային առավելությունների անհրաժեշտությամբ:



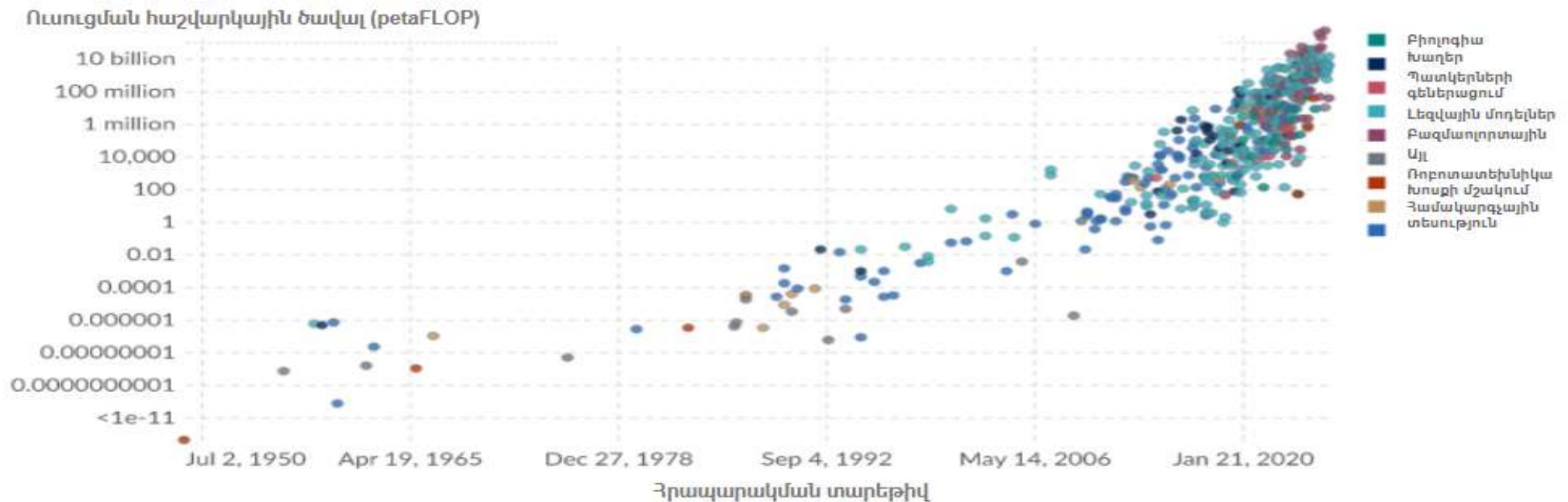
Արհեստական Բանականության Նախադրյալները

Հաշվողական ռեսուրսների աճը ԱԲ համակարգերի ուսուցման մեջ

Ուսուցման հաշվարկային ծավալը չափվում է petaFLOP-ով (10^{15} հաշվարկային գործողություններ):

Տվյալները հիմնված են ԱԲ հետազոտությունների վրա և կարող են ներառել որոշակի շեղում:

Խոշոր՝ օրինակ GPT-4-ի նման մոդելների դեպքում հնարավոր է 2–5-ակի տարբերություն իրական արժեքներից:



Նախնական տվյալներ

Պարամետր	Արժեք	Պարամետր	Արժեք
Նկարագրման լեզու	System Verilog	Ծրագրավորման լեզու	Python, JS
Իրականացման հարթակ	AUP-ZU3 FPGA board	Իրականացման հարթակ	Վեբ կայք
Մուտք/ելք միացումներ	784	Մեծ լեզվային մոդել	LlaMa (3)
LUT	70560	Python modules	FastAPI(Python), OllamaAPI
Տրիգերներ	141120		
Block RAM	216		
DSP	360		
MMCM	3	CPU	11 TH Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz
Board clk frequency	100Mhz		
		GPU	Intel(R) Iris(R) Xe Graphics



Տեսական առնչություն (մաս 1)

ԱԲ Գործակալ (Agentic AI)

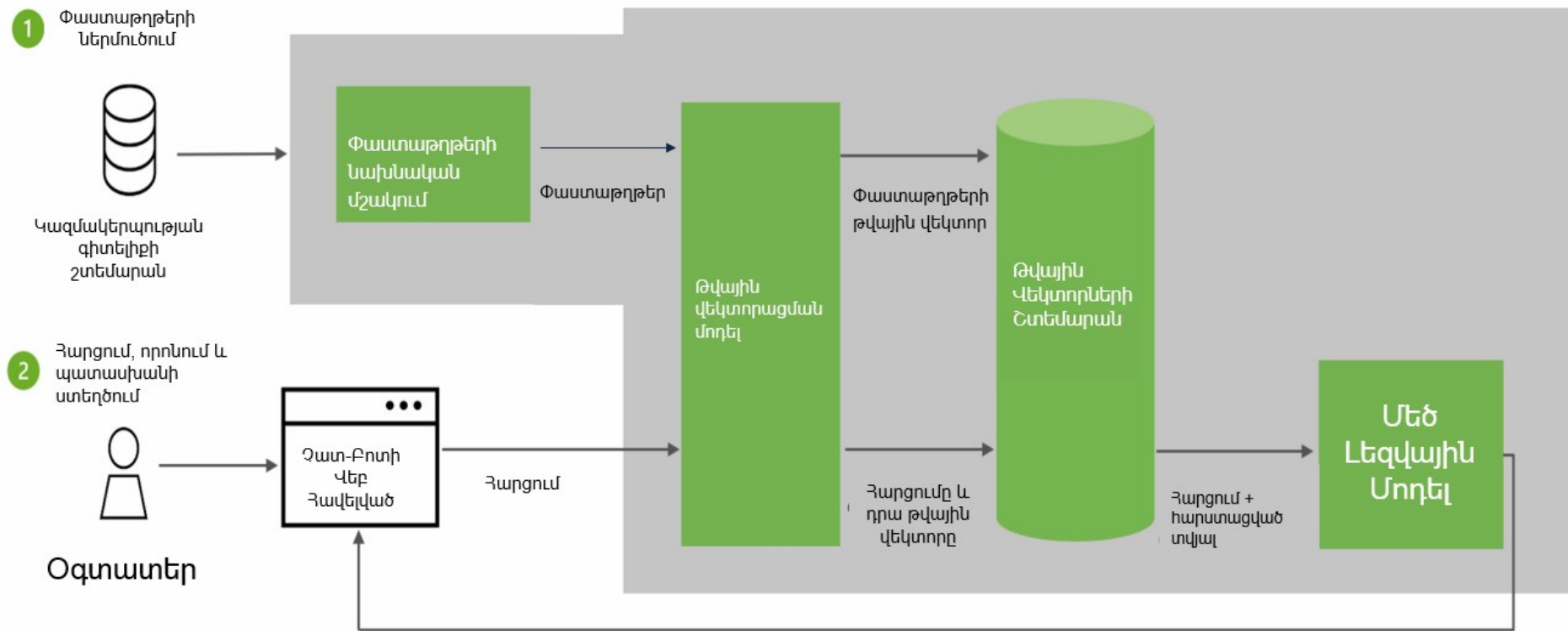
ԱԲ Գործակալը արհեստական բանականությամբ համակարգերի տեսակ է, որը ոչ միայն մշակում է տեղեկատվություն, այլ նաև կատարում է առաջադրանքներ ինքնուրույն ընտրելով քայլերի հաջորդականությունը:

- **Ինքնավարություն** կարողանում են շարունակել գործողությունները առանց մարդու
- **Պլանավորում** գնահատում են հաջորդ քայլերը, ընտրում օպտիմալ լուծումը
- **Գործիքների օգտագործում** որոնում, հաշվարկ, կոդի իրականացում և այլն
- **Հետադարձ կապ** սխալների հայտնաբերում և ուղղում



Տեսական առնչություն (մաս 2)

Տեղեկատվության որոնմամբ հղորացված գեներացիա (RAG)



- R. More, "A Unified Evaluation Framework for Grounded LLM Architectures: Comparative Analysis of RAG, Self-RAG, and Agentic RAG," 2025 5th International Conference on Artificial Intelligence and Signal Processing (AISP), Vijayawada, India, 2025, pp. 1-5, doi: 10.1109/AISP68263.2025.11396270.



Տեսական առնչություն (մաս 3)

Mistral 7B vs Gemma 9B vs Llama3 8B

ՊԱՐԱՄԵՏՐԵՐ

Mistral	7.3B
Gemma 2	9B
Llama 3	8B

ԹՈՂԱՐԿՈՒՄ

Mistral	2023
Gemma 2	2024
Llama 3	2024

CONTEXT

Mistral	32K
Gemma 2	8K
Llama 3	8K

ՃԱՐՏ. ՏԻՊ

Mistral	Dense+GQA
Gemma 2	Dense
Llama 3	Dense+GQA

Կատարողականի Համեմատություն

MMLU **Gemma 2 +**



GSM-8K **Llama 3 +**



MATH **Llama 3 +**



HumanEval **Llama 3 +**

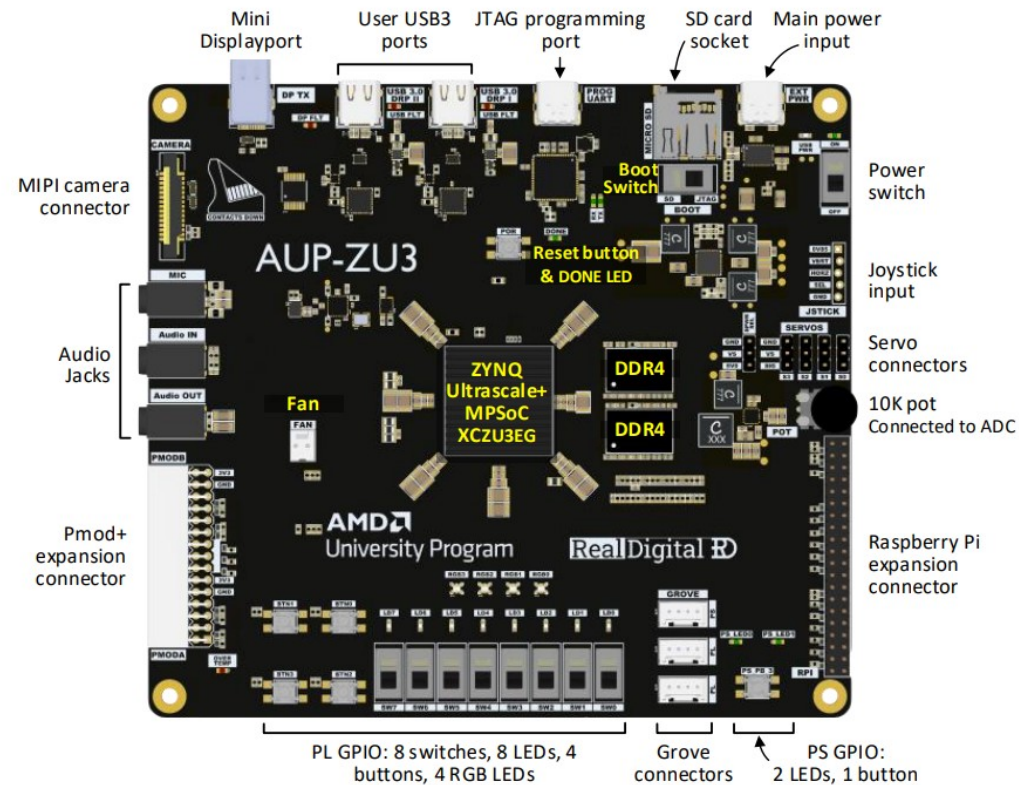


● Mistral 7B ● Gemma 2 9B ● Llama 3 8B



Տեսական առնչություն (մաս 4)

AUP-ZU3 հարթակի կառուցվածքը



Տվյալների քանակի սինթետիկ ավելացում

Մինթետիկ մեթոդով (ԱԲ-ին հարցումներ կատարելու միջոցով) նախնական գրված նկարագրությունների թիվը հասցվել է 16000-ի՝ ի շնորհիվ նրանց հատկանշական փոփոխական պարամետրերի

Դրանց հիմնական շարքում են՝

- Աշխատանքը թույլատրող ազդանշանի առկա լինել/չլինելը
- Տակտային ազդանշանով կառավարումը
- Բիթայնությունը
- Զրոյացնող ազդանշանի առկայությունը և նրա աճող/նվազող ճակատով կառավարումը
- Իրականացման համար ընտրված տրամաբանական տարրերի տեսակը
- Հիշողությունների դեպքում պորտերի քանակը՝ հնարավոր 3 տեսակի (այն պայմանով, որ դրանցից գոնե մեկը ունենա ընթերցելու օպցիա)



Ծրագրային ապահովման մշակում

Backend-ը մշակվել է Python և FastAPI տեխնոլոգիաներով

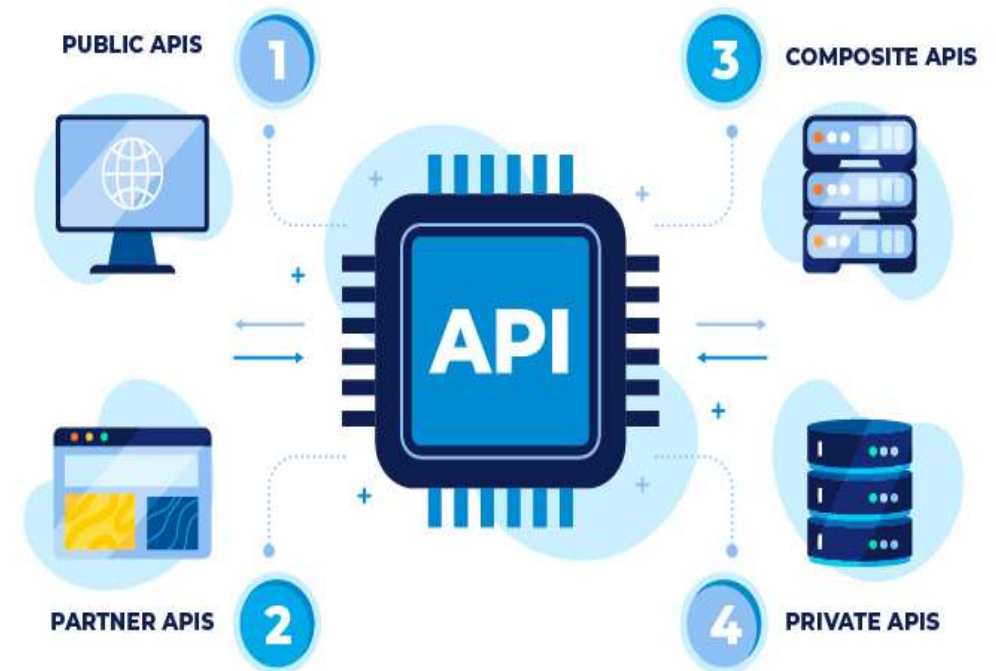
Իրականացվել են տվյալների բեռնում, ինդեքսի վերականգնում և հարցումների մշակում

Ինտեգրվել է տեղային LLM (Ollama, llama3)

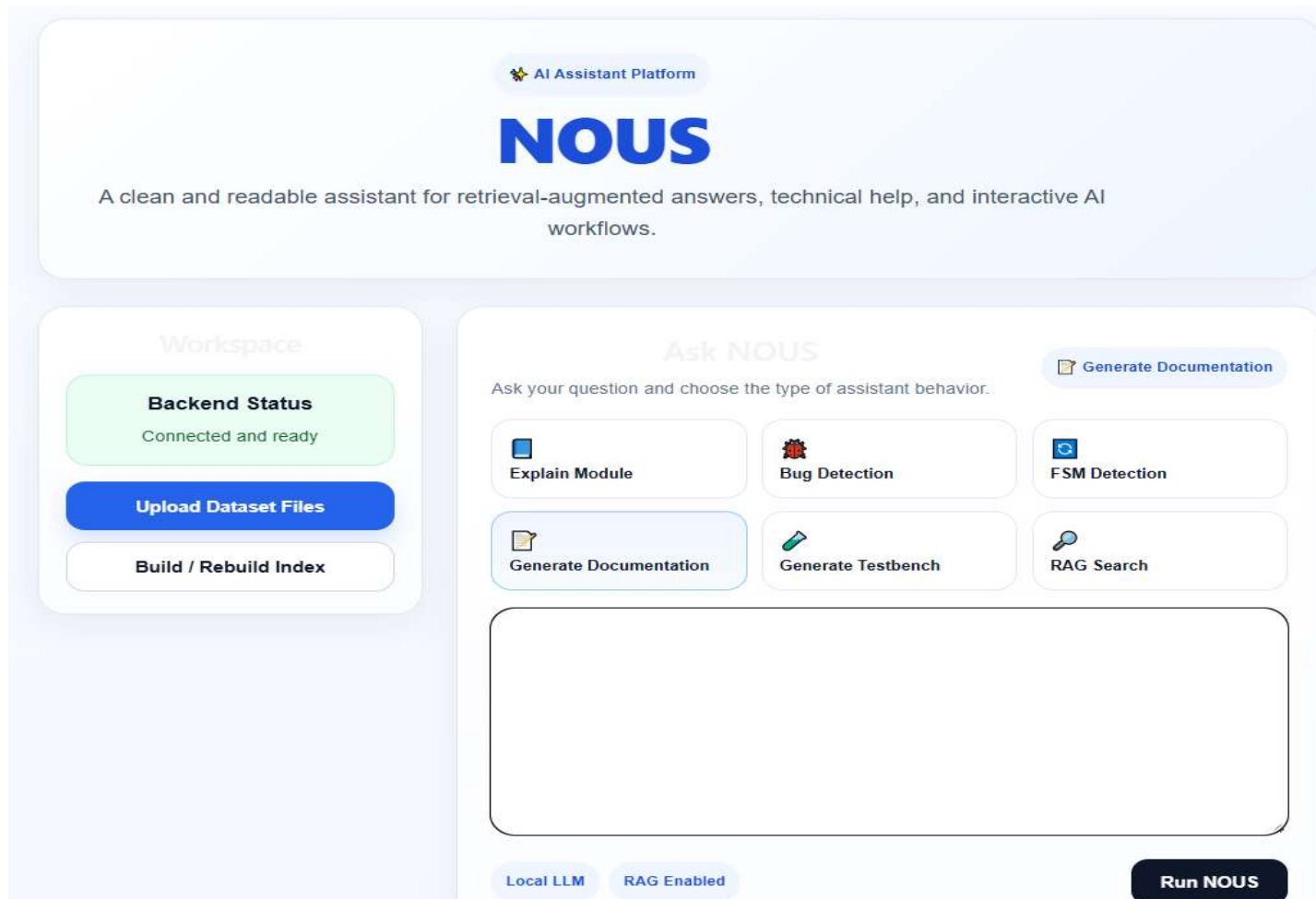
Frontend-ը ստեղծվել է React և Vite միջոցով

Կազմվել է պարզ և հարմար գրույցային միջավայր

Frontend–backend կապը իրականացվել է REST API-ի



Ծրագրային իրականացում (մաս 1)



Ծրագրային իրականացում (մաս 2)

AI Assistant Platform

NOUS

A clean and readable assistant for retrieval-augmented answers, technical help, and interactive AI workflows.

Workspace

Backend Status
Connected and ready

Upload Dataset Files

Build / Rebuild Index

Ask NOUS

Ask your question and choose the type of assistant behavior.

Generate Documentation

Explain Module

Bug Detection

FSM Detection

Generate Documentation

Generate Testbench

RAG Search

Explain the FIFO module in this project and mention possible RTL issues.

Local LLM

RAG Enabled

Run NOUS

Assistant Output

Generate Documentation

****FIFO Module Documentation****

****Purpose:****
The simple_fifo module is a basic implementation of a First-In-First-Out (FIFO) memory block. It allows data to be written into the FIFO through the write enable (wr_en) signal and read from the FIFO through the read enable (rd_en) signal.

****Inputs:****

* `clk`: Clock signal
* `rst`: Reset signal
* `wr\_en`: Write enable signal
* `rd\_en`: Read enable signal
* `din`: Data input

****Outputs:****

* `dout`: Data output
* `full`: Full flag indicating the FIFO is full
* `empty`: Empty flag indicating the FIFO is empty

****Parameters:****

* `DATA\_WIDTH`: The width of each data element (default: 8)
* `DEPTH`: The depth of the FIFO (default: 8)

****Internal Architecture:****
The simple_fifo module consists of a memory array with a specified depth and width, controlled by write and read pointers. When the FIFO is full or empty, the corresponding flags ('full' and 'empty') are set.

****Timing Behavior:****
The FIFO operates on the rising edge of the clock (clk). Writing to the FIFO occurs when wr_en is high, and reading from the FIFO occurs when rd_en is high. The FIFO can be reset by setting rst low.



Ծրագրային իրականացում (մաս 3)

AI Assistant Platform

NOUS

A clean and readable assistant for retrieval-augmented answers, technical help, and interactive AI workflows.

Workspace

Backend Status

Connected and ready

Upload Dataset Files

Build / Rebuild Index

Ask NOUS

Ask your question and choose the type of assistant behavior.

Generate Testbench

Explain Module

Bug Detection

FSM Detection

Generate Documentation

Generate Testbench

RAG Search

Generate a Verilog testbench for the memory_controller module that verifies read, write, reset, and ready signal behavior.

Local LLM

RAG Enabled

Running...

```
// Reset the memory controller
#(100); // wait for 100 clock cycles
rst_n = 1; // deassert reset

// Test read and write operations
@(posedge clk);
read_en = 1;
addr = 2;
write_data = 5;
#(10); // wait for 10 clock cycles
assert(read_data == 5) $display("Read
data: %d", read_data);

@(posedge clk);
write_en = 1;
addr = 3;
write_data = 7;
#(10); // wait for 10 clock cycles
assert(read_data == 7) $display("Read
data: %d", read_data);

// Test reset behavior
@(posedge clk);
rst_n = 0; // assert reset
#(100); // wait for 100 clock cycles
rst_n = 1; // deassert reset

// Verify ready signal behavior
@(posedge clk);
read_en = 1;
addr = 4;
write_data = 3;
#(10); // wait for 10 clock cycles
assert(ready == 1) $display("Ready
```

Here is a useful Verilog testbench for the memory_controller module:

File: tb_memory_controller.v

```
***verilog
// Testbench for Memory Controller
module tb_memory_controller;

    parameter DATA_WIDTH = 8;
    parameter ADDR_WIDTH = 4;
    parameter MEM_DEPTH = 16;

    reg clk;
    reg rst_n;
    reg read_en;
    reg write_en;
    reg [ADDR_WIDTH-1:0] addr;
    reg [DATA_WIDTH-1:0] write_data;
    wire [DATA_WIDTH-1:0] read_data;
    wire ready;

    // Instantiate the memory controller
    memory_controller #(
        .DATA_WIDTH(DATA_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH),
        .MEM_DEPTH(MEM_DEPTH)
    ) uut (
        .clk(clk),
        .rst_n(rst_n),
        .read_en(read_en),
        .write_en(write_en),
        .addr(addr),
        .write_data(write_data),
        .read_data(read_data),
        .ready(ready)
    );
```



Արդյունքների համեմատում ըստ փուլերի(մաս 1)

Պարզ Սխեմաներ

ռեգիստրներ

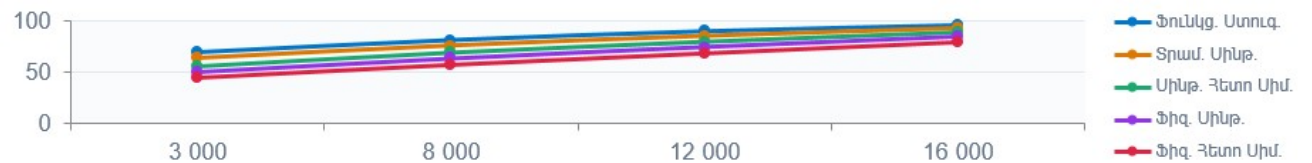
հաշվիչներ

կոդավորիչներ

կոդի փոխակերպիչներ

Միջին ճշգրտությունը՝ 88.66%

Փուլ	3 000	8 000	12 000	16 000
Ֆունկց. Ստուգ.	70.2%	81.8%	90.2%	96%
Տրամ. Սինթ.	63.8%	76.2%	85.8%	93.5%
Սինթ. Չեղո Սիմ.	56.8%	69.8%	80.2%	89.2%
Ֆիզ. Սինթ.	50.8%	63.8%	74.8%	84.8%
Ֆիզ. Չեղո Սիմ.	44.8%	57.5%	68.8%	79.8%



Արդյունքների համեմատում ըստ փուլերի(մաս 2)

FIFO/LIFO

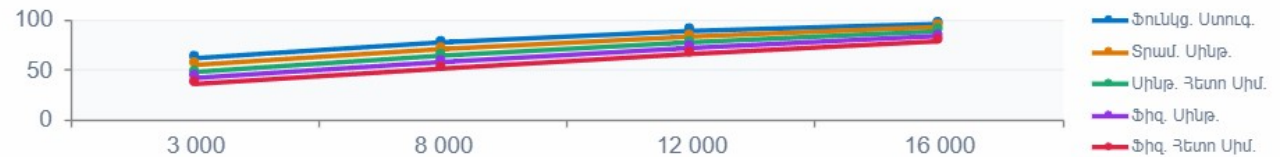
վերջավոր
ավտոմատներ

թվաբանական
սխեմաներ

Միջին ճշգրտությունը՝ 87.38%

Միջին Բարդ. Սխեմաներ

Փուլ	3 000	8 000	12 000	16 000
Ֆունկց. Ստուգ.	61%	77%	88.3%	95.3%
Տրամ. Սինթ.	54.3%	70.3%	82.7%	92.3%
Սինթ. Հետո Սիմ.	47.3%	63.7%	77%	88%
Ֆիզ. Սինթ.	41.3%	57.3%	71.3%	83.3%
Ֆիզ. Հետո Սիմ.	35.3%	50.7%	65.3%	78%



Արդյունքների համեմատում ըստ փուլերի(մաս 3)

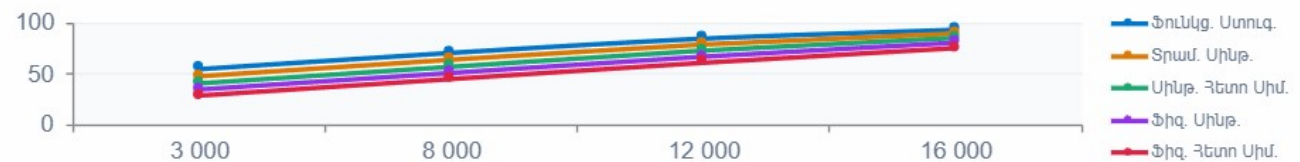
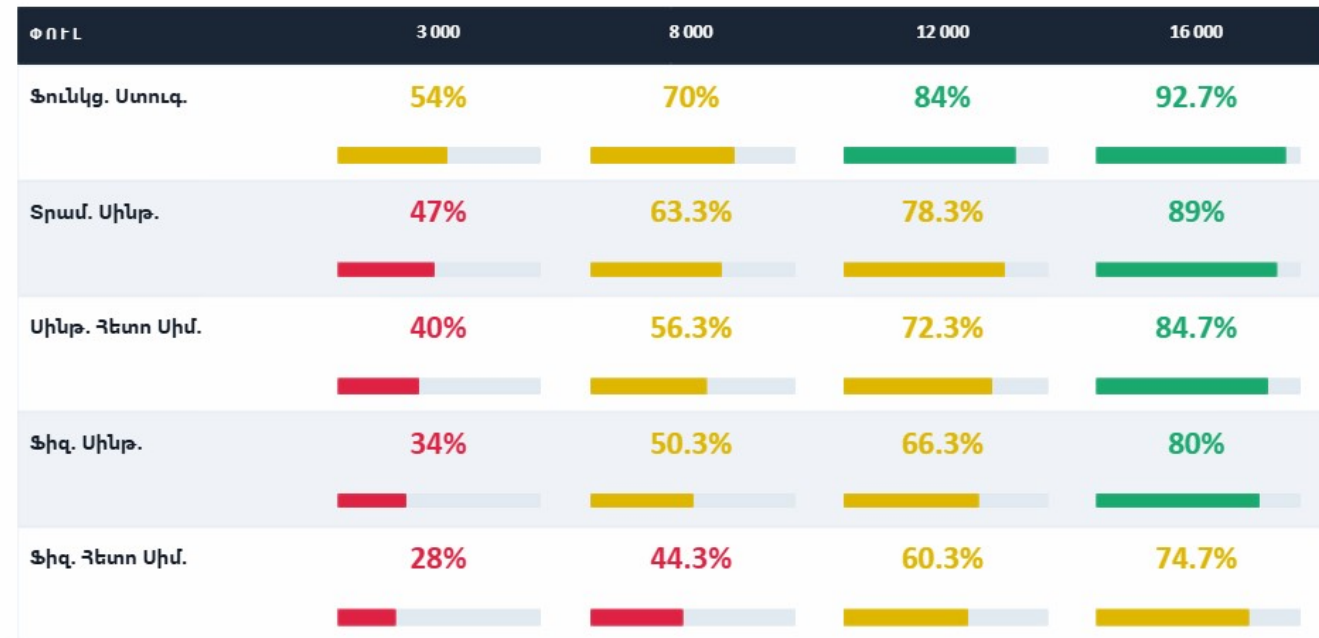
Բարդ Սխեմաներ

հիշողություններ

ինտերֆեյսեր

հիշողության
կառավարիչներ

Միջին ճշգրտությունը՝ 84.08%



ՍԽԱԼՆԵՐԻ ՊԱՏՃԱՌՆԵՐԻ ԸՆԴՀԱՆՈՒՐ ԴԱՍԱԿԱՐԳՈՒՄ

ՊԱՐԶ

- Timing failure
- Clock-ի սխալ ճակատ
- LUT-ի ծավալի գերազանցում
- Bitwidth failure
- Սխալ ձևով գրված always բլոկ
- Սխալ reset

ՄԻՋԻՆ

- Անորոշ վիճակների առկայություն
- Full/Empty flag
- 0-ի բաժանում
- Հնարավորություններից ավելի բարդ համակցություններ
- Երկար կապուղիներ
- W/R conflict

ԲԱՐԴ

- Read latency
- Handshake error
- Տվյալների կորուստ
- Multi-master սխալ կառավարում
- Չափազանց բարդ կառավարման տրամաբանություն
- LUT/LUTRAM-երի սխեմայում RAM-երի փոխարեն



Արդյունքների ամփոփում

NOUS ծրագրային օգնականի արդյունքները

86.73%

Անցման մակարդակ

NOUS-ն ապահովել է բարձր անցման մակարդակ AUP-ZU3-ի համար թվային սխեմաների գեներացման, ֆունկցիոնալ ստուգման, սինթեզի և սիմուլյացիայի գործընթացներում, ցուցաբերելով կայուն և կանխատեսելի վարքագիծ հատկապես բարդ կառուցվածքների դեպքում:

1 Պարզ սխեմաներ

CLAUDE Sonnet free	93%
ChatGPT-4o mini free	91%
NOUS	89%
DeepSeek-V3 free	83%

2 Միջին բարդության սխեմաներ

CLAUDE Sonnet free	88%
NOUS	87%
ChatGPT-4o mini free	85%
DeepSeek-V3 free	73%

3 Բարդ սխեմաներ

NOUS	84%
CLAUDE Sonnet free	80%
ChatGPT-4o mini free	72%
DeepSeek-V3 free	64%

Claude-ի հավասար
Ընդհանուր անցման մակարդակով

ChatGPT-ից բարձր
Միջին բարդության սխեմաների դեպքում

Ամենաբարձրը
Բարդ սխեմաների դեպքում

Արդյունքների ամփոփում

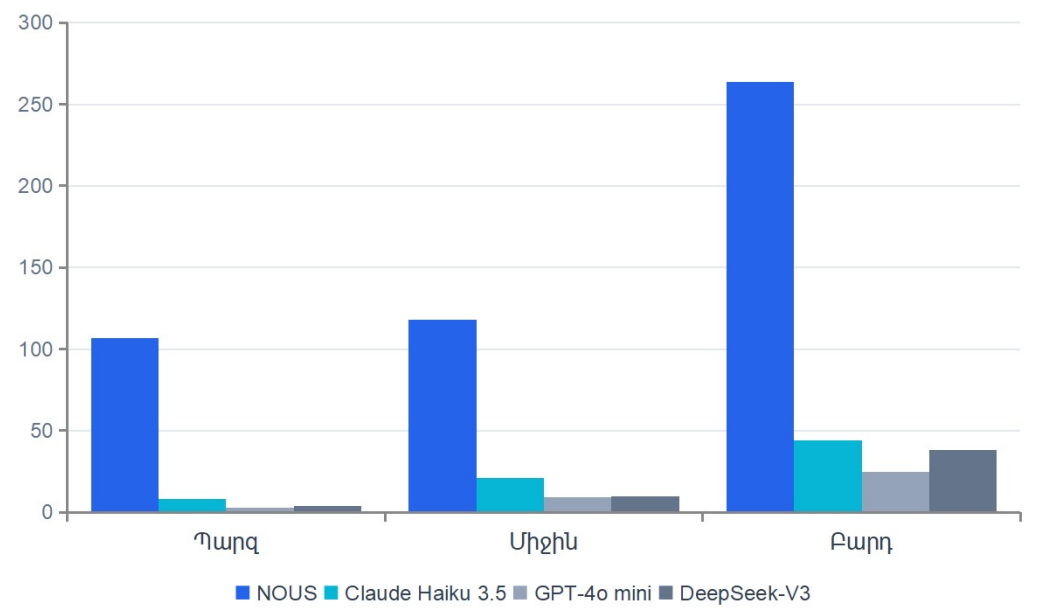
NOUS — Դիզայն · Հարմարավետություն · Ժամանակային համեմատություն

86.73%

Անցման մակարդակ

Ինտերֆեյս	
Չափանիշ	NOUS
Դիզայն	Մինիմալ, մաքուր
Հատուկ կոճակներ	✓ 6 mode
Backend կարգ.	✓ Ցուցադրված
RAG / Local LLM	✓ Ակտիվ
Թեմատիկ ինտերֆեյս	✓ EDA/HDL
Կոնֆիդենցիալություն	Local / RAG

Սխեմաների Տրամադրման Ժամանակ (վարկյան)



NOUS Պարզ: 107 վ.

NOUS Միջին: 118 վ.

NOUS Բարդ: 264 վ.

Դիզայն

EDA/HDL-ին հատուկ ինտերֆեյս մաքուր ու կենտրոնացված

Ֆունկցիոնալություն

6 հատուկ mode, Backend կարգավիճակ, Index կառ.

Գաղտնիություն

Local LLM + RAG — տվյալները դուրս չեն գալիս

ԵԶՐԱԿԱՑՈՒԹՅՈՒՆ

- Նախագծվել և իրականացվել է ինտեգրալ սխեմաների նախագծման օգնական՝ NOUS անվանումով, որը հնարավորություն է տալիս գեներացնել թվային սխեմաների System Verilog նկարագրությունները և դրանց համապատասխան ստուգող փորձարկման միջավայրերը AUP-ZU3 հարթակի համար:
- Տվյալ օգնականով գեներացված սխեմաները ունեն 86.73% միջին ճշգրտություն: Առկա մեթոդների համեմատած ընդհանուր միջին ճշգրտության դասը մեծամասնությունից բարձր է, իսկ բարդ սխեմաների դեպքում առաջատարն է, քանզի բարդ սխեմաների օպտիմալացմանը ավելի մեծ ռեսուրսներ և ժամանակ է տրամադրվել՝ այդ սխեմաների ԻՍ-երում կարևորության և կիրառման առաջնահերթության պատճառով:
- Ծրագրային օգնականի ընդհանուր ինտերֆեյսը իր հարմարավետությամբ հարմարեցված է հատուկ նեղ մասնագիտական շրջանակներում օգտագործման համար:
- Հետևյալ ծրագրային օգնականի համար որպես հեռանկար կարելի է դիտարկել նրա կիրառման շրջանակների ընդլայնումը, ինտերֆեյսի զարգացումը, ինչպես նաև արդյունքների տրամադրման ժամանակի կրճատումը:



ՀԱՌԱՅԱԿԱԼՈՒԹՅՈՒՆ